Prompt Optimizer

Multi-Model Arena for Autonomous Prompt Refinement

Molinari Lorenzo

The problem

The "Prompt Engineering" Bottleneck

The Core Problem

- Prompt Engineering is Brute Force: Developers spend hours manually iterating on prompts ("add this constraint," "change tone," "fix edge case").
- Scalability Bottleneck: Human intuition is slow and biased. We can't manually optimize prompts for every new model or domain.
- The "Black Box" Issue: We don't always know **why** a prompt fails until we test it.

My Re-framing
Instead of just "rewriting" a prompt, we treat prompt engineering as an optimization search problem.

- Search Space: The infinite combinations of words.
- Objective Function: A strict evaluation score (0-10) based on clarity, robustness, and constraints.
- Optimizer: An LLM (Mistral) that acts as both the *generator* of variations and the *judge* of quality.
- Structure: A Refinement Tree where each node is a prompt version, and we prune branches with low scores.

The solution

“Arena” Architecture

Conceptual Architecture -> The "Arena" Approach
This architecture moves beyond single attempts by utilizing a Multi-Model Arena to choose the best prompt on every iteration.

1. Draft Input: User provides a rough, natural language intent.
2. Generation Phase
   `mistral-small`: Fast, concise variations.
   `mistral-medium`: Balanced, reasoning-medium level.
   `mistral-large`: High, heavy reasoning.
   `codestral`: Specialized syntax optimizations (only for SWE requests).
3. Evaluation Phase (The Judge)
   mistral-large`: Acts as the harsh critic.
   Strict Criteria: Penalties for vagueness, lack of structure, or missing edge cases.
   Score: 0-10 scale.
4. Selection & Iteration
   The best candidate becomes the "seed" for the next generation.
   Loop continues until Score > 9.8 or Max Iterations reached.

Demo “General”

Prompt Refiner

Turn rough prompts into powerful instructions

Rough Prompt

Identify a book by its quote

Context (Optional)

Literary Assistant. Identify the book, author, and year. Explain the quote's significance

Domain

General

Iterations

3

Refine Prompt

Demo “Code”

Prompt Refiner

Turn rough prompts into powerful instructions

Rough Prompt

Identify a book by its quote

Context (Optional)

Literary Assistant. Identify the book, author, and year. Explain the quote's significance

Domain

General

Iterations

3

Refine Prompt

Implementation

Technical explanation

Key Technical Components

Backend: `FastAPI` (Async/Await for parallel model calls).
Reasoning Engine: `Mistral AI API` (Orchestrating different models).
State Management: `SQLModel` (SQLite) for tracking the tree of prompts.
Frontend: Vanilla JS/CSS/HTML, simple frontend.

Implementation

- **The Iterative Loop** (`refiner.py`): the heartbeat of the system. It's not a single call but a loop, in every iteration, we spawn concurrent futures to get variations from different models, then select the max score.
- **The Judge** (`_evaluate_prompt`): evaluation phase, prompt mistral Large to be strict evaluating results.

Conclusions

Final overview

What's Working
- Self-Correction: the system tries to catch its own mistakes and vagueness to build a better prompt.
- Domain Switching: routing coding tasks to `codestral` to improve coding proficiency.

What could be optimized
- Judge Bias: maybe `mistral-large` evaluating self generated text can have a self-preference bias (not tested so much).
- Local Optima: sometimes it gets stuck polishing a "good enough" structure instead of trying a different approach.
- Cost/Latency: serial iterations take much time and tokens (since the size of prompt increase).

Future directions
- Merging algorithms: implement crossover (combining parts of two top prompts).
- Learning Dataset: saving the best prompts and use them to compare and build the new ones.
- Human-in-the-loop Feedback: allow the user to "nudge" the direction of the result.
- Decision algorithm: tree algorithm to choose is good for a simple version but maybe the direction is to go in the direction of probabilistic models, like implementing Bayesian networks.